ROB521 Lab 2 Report

Claire Zhang, 1008001822
Trent Rossos, 1008073914
Andreas Arzoumanian,
Nayaab Ali, 1008013917

Table of Contents & List of Figures

Part A **2**
RRT 2
Figure 1: A picture with a successful output from l2_planning.py with RRT 3
RRT* 3
Figure 2: A picture with a successful and improved output from l2_planning.py with RRT* 4
Table 1: RRT (Left) VS RRT* (Right) Results 5
Video 5
Figure 3: A video with a successful run of trajectory rollout on the path from RRT on willowgarage map 5
Part B **5**
Simulation Video 5
Figure 4: A video with a successful run of trajectory rollout on the path from RRT on Myhal map 5
In Lab Video 5
Figure 4: A video with a successful run of trajectory rollout on the path from RRT or RRT* on Myhal map 6
Code **7**
l2_planning.py 7
l2_follow_path.py 17
l2_follow_path_myhal.py 23

Part A

RRT

The basic structure of the RRT algorithm can be broken into a few distinct steps, (1) randomly sample a point in free space, (2) identify the closest node in the tree, (3) calculate what control input is needed to steer robot towards that point and enact it, (4) Create a new node at end of trajectory, (5) continue until goal state is reached. Within these steps the most difficult task was being able determine and simulate the imputed control. The *simulate_trajectory* function is used to determine the control input needed to move the robot towards the goal, the corresponding trajectory is then calculated and converted into the global frame. The control input is calculated by first determining the distance and desired heading to get to the goal pose, these measurements are used to determine the differences between the current pose and target pose, which are fed into a basic proportional controller. Using these generated velocities, *trajectory_rollout* uses the unicycle robot model to extrapolate the robot's positions. The model was discretized such that the robot's next position is the sum of its current position and the input velocity multiplies by the length of the timestep. Finally, the trajectory is currently in the robot's frame, so they are transformed to the global frame using a simple 2D rotation matrix with an additional translational shift. Now, the RRT function uses the generated trajectory to create a new node at the end of the motion, checks if the goal is reached and if not the process is repeated.

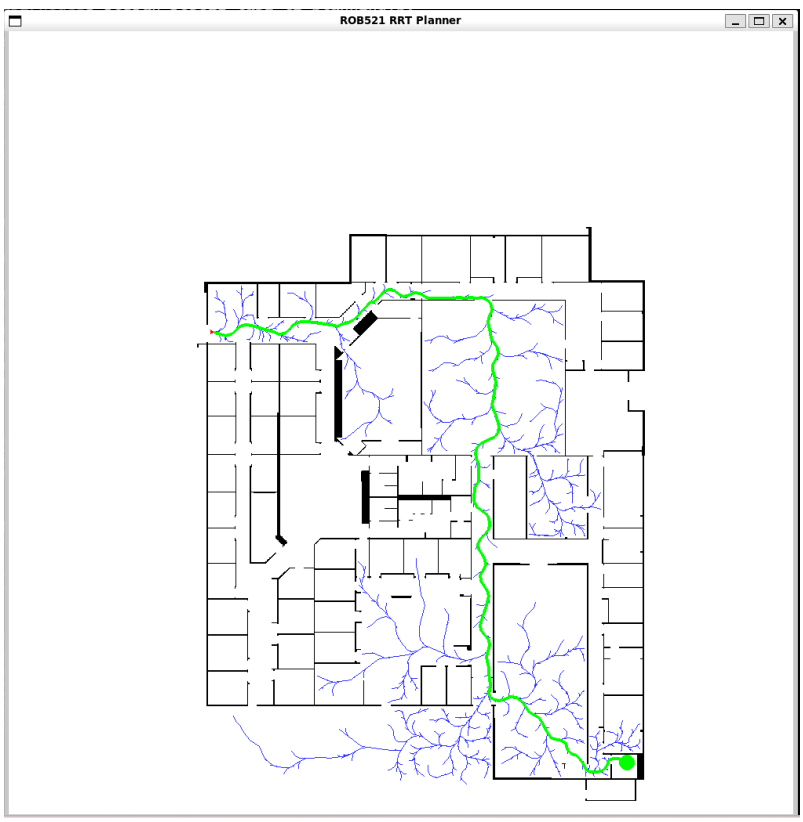

Figure 1: A picture with a successful output from l2_planning.py with RRT

RRT*

RRT* is an asymptotically optimal extension of RRT. Rather than always connecting to the single nearest node, RRT* considers all nodes within a specified radius and selects as parent the one that yields the lowest path cost. Unlike standard RRT, it can also update existing connections by checking whether routing through the new node improves the cost of nearby nodes and changing the parent node if true. This rewiring process gradually improves the tree toward an optimal path through the maze, at the expense of higher computational cost compared with basic RRT.

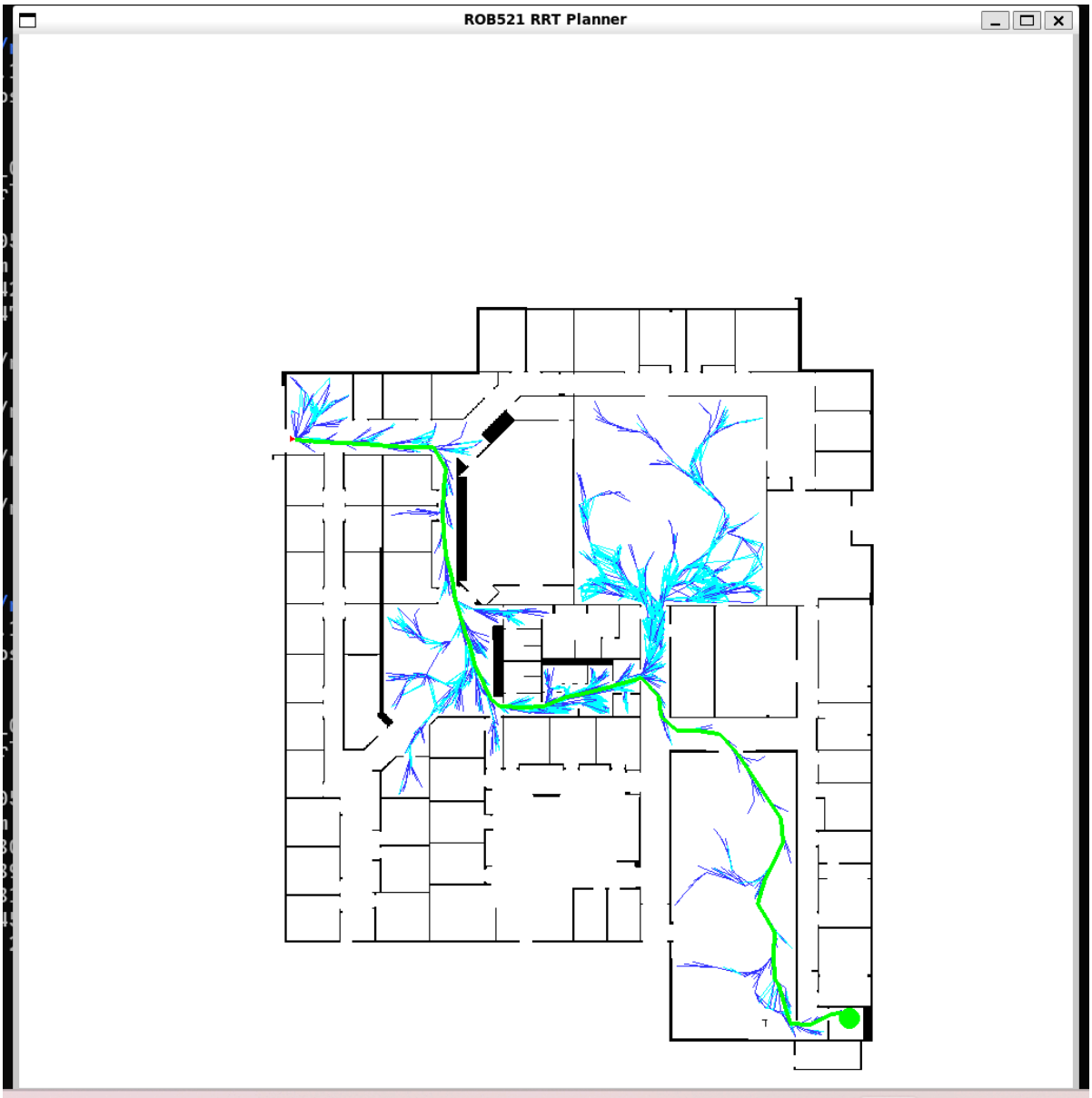

*Figure 2: A picture with a successful and improved output from l2_planning.py with RRT**

Improvements from RRT versus RRT* shown in Table 1 below. RRT* had less iterations and shorter path distance (in meters), though it was slower to complete (in seconds), in comparison to RRT.

RRT Output	**RRT* Output**
Iter 0 | Nodes: 2 | Dist: 60.37m Iter 2500 | Nodes: 247 | Dist: 52.17m Iter 7500 | Nodes: 1079 | Dist: 21.37m Iter 8000 | Nodes: 1282 | Dist: 23.22m Iter 9000 | Nodes: 1614 | Dist: 46.62m Iter 10500 | Nodes: 2234 | Dist: 14.41m Iter 11000 | Nodes: 2413 | Dist: 39.73m Iter 12500 | Nodes: 2874 | Dist: 2.31m Goal reached at iteration 12511! Path saved. Duration: 73.74s Total Path Distance: 93.60 meters	Iter 0 | Nodes: 2 | Goal Dist: 60.37m Iter 1500 | Nodes: 256 | Goal Dist: 30.30m Iter 4500 | Nodes: 511 | Goal Dist: 39.14m Iter 5000 | Nodes: 548 | Goal Dist: 31.45m Iter 7500 | Nodes: 942 | Goal Dist: 45.95m Iter 9500 | Nodes: 1297 | Goal Dist: 29.77m Goal reached at iteration 11256! Path saved. Duration: 78.54s Total Path Distance: 79.63 meters

Table 1: RRT (Left) VS RRT (Right) Results.*

Video

Figure 3 is a screenshot of the attached video titled RRT_willowgarage.mp4, showing l2_follow_path.py capabilities based on the .npy file generated by l2_planning. Google Drive viewing link here.

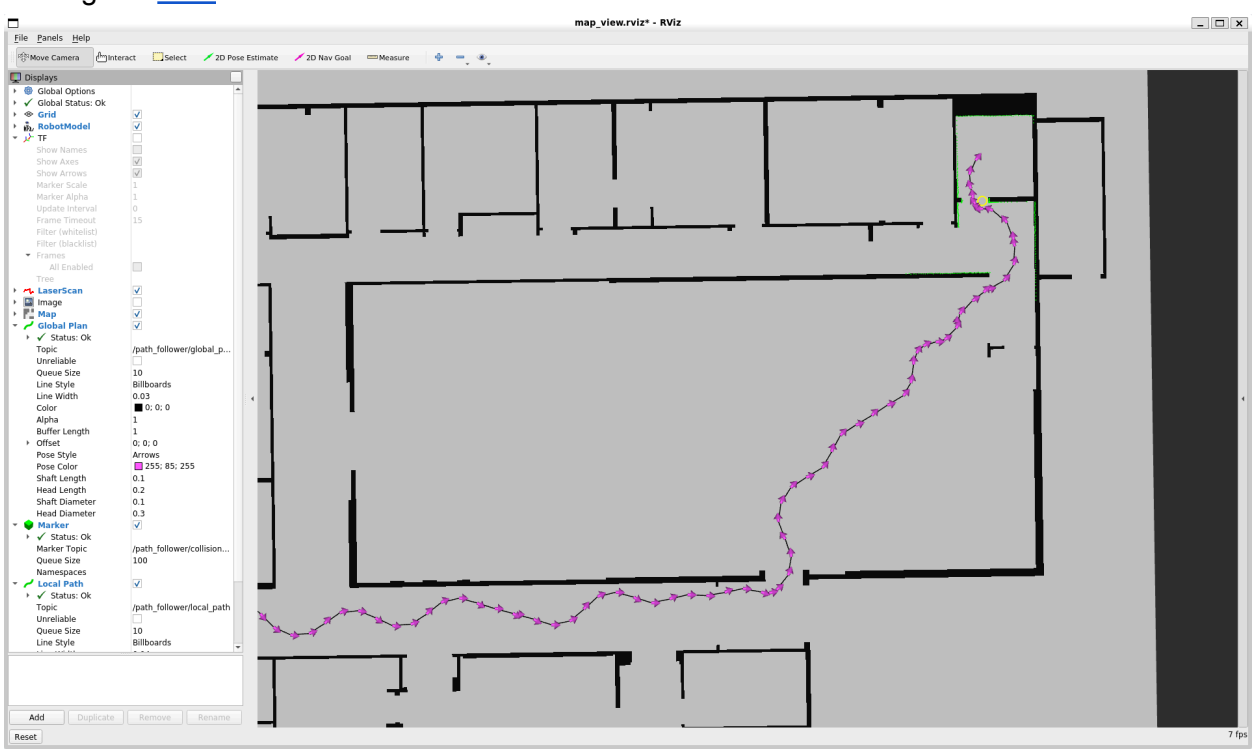

Figure 3: A video with a successful run of trajectory rollout on the path from RRT on willowgarage map.

Part B

Simulation Video

Figure 4 is a screenshot of the attached video titled RRT_myhal.mp4. Google drive viewing link available here.

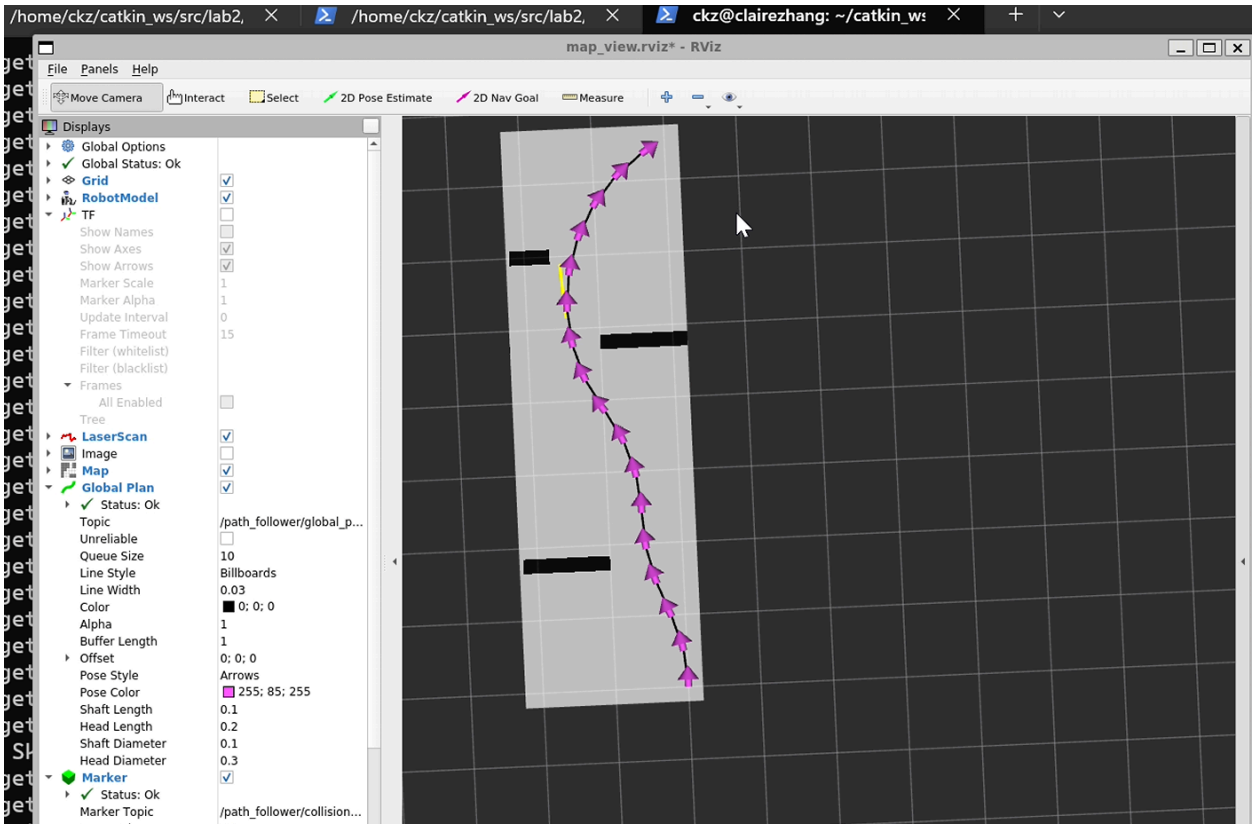

Figure 4: A video with a successful run of trajectory rollout on the path from RRT on Myhal map.

In Lab Video

The video of the completed in-lab demo can be viewed here.

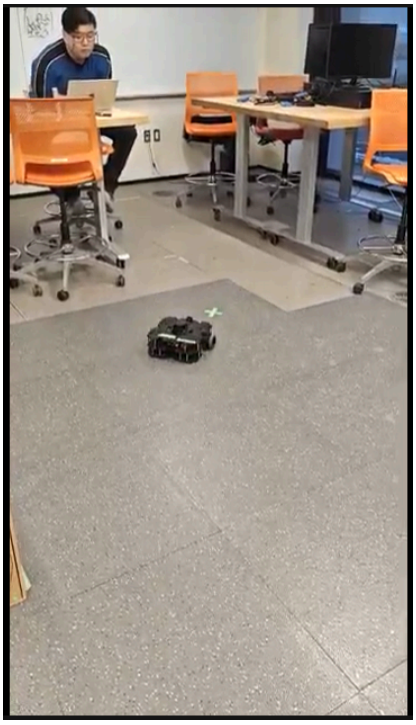

Figure 4: A video with a successful run of trajectory rollout on the path from RRT or RRT on Myhal map.*

Code

l2_planning.py

```
#!/usr/bin/env python3
# Standard Libraries
import numpy as np
import yaml
import pygame
import time
import pygame_utils
import matplotlib.image as mpimg
from skimage.draw import disk
from scipy.linalg import block_diag


def load_map(filename):
    im = mpimg.imread("../maps/" + filename)
    if len(im.shape) > 2:
        im = im[:,:,0]
    im_np = np.array(im)  #Whitespace is true, black is false
    #im_np = np.logical_not(im_np)
    return im_np


def load_map_yaml(filename):
    with open("../maps/" + filename, "r") as stream:
            map_settings_dict = yaml.safe_load(stream)
    return map_settings_dict

#Node for building a graph
class Node:
    def __init__(self, point, parent_id, cost):
        self.point = point # A 3 by 1 vector [x, y, theta]
        self.parent_id = parent_id # The parent node id that leads to this
node (There should only every be one parent in RRT)
        self.cost = cost # The cost to come to this node
        self.children_ids = [] # The children node ids of this node
        return
```

```
# Path Planner
class PathPlanner:
    #A path planner capable of perfomring RRT and RRT*
    def __init__(self, map_filename, map_settings_filename, goal_point,
stopping_dist):
        print("-" * 30) ##added during testing to make it easier to find
the start of the planner in the terminal output
        #Get map information
        self.occupancy_map = load_map(map_filename)
        self.map_shape = self.occupancy_map.shape
        self.map_settings_dict = load_map_yaml(map_settings_filename)

        #Get the metric bounds of the map
        self.bounds = np.zeros([2,2]) #m
        self.bounds[0, 0] = self.map_settings_dict["origin"][0]
        self.bounds[1, 0] = self.map_settings_dict["origin"][1]
        self.bounds[0, 1] = self.map_settings_dict["origin"][0] +
self.map_shape[1] * self.map_settings_dict["resolution"]
        self.bounds[1, 1] = self.map_settings_dict["origin"][1] +
self.map_shape[0] * self.map_settings_dict["resolution"]

        # Global Bounding Box (so it doesn't sample outside the maze -
added during testing to speed up sampling)
        self.min_x, self.max_x = -1.0, 44.0
        self.min_y, self.max_y = -46.0, 5.0

        #Robot information
        self.robot_radius = 0.22 #m
        self.vel_max = 0.6 #m/s (Feel free to change!) ##changed
        self.rot_vel_max = 0.5 #rad/s (Feel free to change!) ##changed

        # Goal Parameters
        self.goal_point = goal_point
        self.stopping_dist = stopping_dist

        # Trajectory Simulation Parameters
        self.timestep = 1.0
        self.num_substeps = 10

        # Planning storage (Start at 0,0,0)
```

```
        self.nodes = [Node(np.zeros((3, 1)), -1, 0)]

        #RRT* Specific Parameters
        self.lebesgue_free = np.sum(self.occupancy_map) *
self.map_settings_dict["resolution"] **2
        self.zeta_d = np.pi
        self.gamma_RRT_star = 2 * (1 + 1/2) ** (1/2) * (self.lebesgue_free
/ self.zeta_d) ** (1/2)
        self.gamma_RRT = self.gamma_RRT_star + .1
        self.epsilon = 2.5

        #Pygame window for visualization
        self.window = pygame_utils.PygameWindow(
            "Path Planner", (1000, 1000), self.occupancy_map.shape,
self.map_settings_dict, self.goal_point, self.stopping_dist)
        return

    def sample_map_space(self):
        #Return an [x,y] coordinate to drive the robot towards
        #print("TO DO: Sample point to drive towards")

        ## Added some bias to sample in which direction to expand the tree
        # Goal Bias 20% of time - look in direction of the goal to speed
up convergence
        if np.random.rand() < 0.20:
            return self.goal_point

        # Adaptive Local Window 60% of time - look only in a local window
around the best node to speed up convergence while still allowing some
exploration
        if np.random.rand() < 0.60:
            dists_to_goal = [np.linalg.norm(n.point[0:2] -
self.goal_point) for n in self.nodes]
            best_pos =
self.nodes[np.argmin(dists_to_goal)].point[0:2].flatten()

            # Closer to goal = smaller window
            dist_to_goal_val = np.min(dists_to_goal)
            window_size = 5.0 if dist_to_goal_val < 5.0 else 20.0
```

```
            x = np.random.uniform(max(self.min_x, best_pos[0] -
window_size/2),
                                  min(self.max_x, best_pos[0] +
window_size/2))
            y = np.random.uniform(max(self.min_y, best_pos[1] -
window_size/2),
                                  min(self.max_y, best_pos[1] +
window_size/2))
            return np.array([[x], [y]])

        # Global Random Exploration 20% - sample anywhere in the map to
ensure we don't miss any narrow passages
        x = np.random.uniform(self.min_x, self.max_x)
        y = np.random.uniform(self.min_y, self.max_y)
        return np.array([[x], [y]])

    def check_if_duplicate(self, point):
        # #Check if point is a duplicate of an already existing node
        # print("TO DO: Check that nodes are not duplicates")

        if not self.nodes:
            return False

        # Calculate the distance to nearest neighbor
        idx = self.closest_node(point)
        closest_coords = self.nodes[idx].point[0:2]
        dist = np.linalg.norm(closest_coords - point)
        if dist < 1e-3:
            return True
        else:
            return False

    def closest_node(self, point):
        # #Returns the index of the closest node
        # print("TO DO: Implement a method to get the closest node to a
sapled point")

        if not self.nodes:
            return None
```

```
        dists = []
        for node in self.nodes:
            # Calculate distance using only X and Y (indices 0 and 1)
            node_coords = node.point[0:2]
            d = np.linalg.norm(node_coords - point)
            dists.append(d)

        return np.argmin(dists)


    def simulate_trajectory(self, node_i, point_s):
        # #Simulates the non-holonomic motion of the robot.
        # #This function drives the robot from node_i towards point_s.
This function does has many solutions!
        # #node_i is a 3 by 1 vector [x;y;theta] this can be used to
construct the SE(2) matrix T_{OI} in course notation
        # #point_s is the sampled point vector [x; y]
        # print("TO DO: Implment a method to simulate a trajectory given a
sampled point")
        vel, rot_vel = self.robot_controller(node_i, point_s)

        robot_traj = self.trajectory_rollout(vel, rot_vel, node_i)
        return robot_traj

    def robot_controller(self, node_i, point_s):
        #This controller determines the velocities that will nominally
move the robot from node i to node s
        #Max velocities should be enforced
        # print("TO DO: Implement a control scheme to drive you towards
the sampled point")
        # return 0, 0

        xi, yi, thi = node_i.flatten()

        # Calculate distance and angle to target
        dx = point_s[0, 0] - xi
        dy = point_s[1, 0] - yi
        dist = np.hypot(dx, dy)

        # Determine the steering error (alpha)
```

```
        target_th = np.arctan2(dy, dx)
        alpha = (target_th - thi + np.pi) % (2 * np.pi) - np.pi

        # Set velocities based on distance and angle
        vel = max(0.1, min(self.vel_max, 0.5 * dist))
        rot_vel = np.clip(3.0 * alpha, -self.rot_vel_max,
self.rot_vel_max)

        return vel, rot_vel

    def trajectory_rollout(self, vel, rot_vel, node_i):
        # Given your chosen velocities determine the trajectory of the
robot for your given timestep
        # The returned trajectory should be a series of points to check
for collisions
        # print("TO DO: Implement a way to rollout the controls chosen")
        # return np.zeros((3, self.num_substeps))

        traj = np.zeros((3, self.num_substeps))
        dt = self.timestep / self.num_substeps
        curr = node_i.flatten().astype(float)

        for i in range(self.num_substeps):
            # Update X, Y, and Theta (Heading)
            curr[0] += vel * np.cos(curr[2]) * dt
            curr[1] += vel * np.sin(curr[2]) * dt
            curr[2] += rot_vel * dt

            traj[:, i] = curr

        return traj

    def point_to_cell(self, point):
        #Convert a series of [x,y] points in the map to the indices for
the corresponding cell in the occupancy map
        #point is a 2 by N matrix of points of interest
        # print("TO DO: Implement a method to get the map cell the robot
is currently occupying")
        # return 0
```

```
        res = self.map_settings_dict["resolution"]
        orig = self.map_settings_dict["origin"]
        point = np.array(point)

        x = point[0, :]
        y = point[1, :]

        # Convert to Pixel Coordinates, col corresponds to x, row
corresponds to y (inverted)
        col = ((x - orig[0]) / res).astype(int)
        row = (self.map_shape[0] - 1 - (y - orig[1]) / res).astype(int)

        # Bounds
        col = np.clip(col, 0, self.map_shape[1] - 1)
        row = np.clip(row, 0, self.map_shape[0] - 1)

        return row, col

    def points_to_robot_circle(self, points):
        # #Convert a series of [x,y] points to robot map footprints for
collision detection
        # #Hint: The disk function is included to help you with this
function
        # print("TO DO: Implement a method to get the pixel locations of
the robot path")

        res = self.map_settings_dict["resolution"]
        rows, cols = self.point_to_cell(points)
        radius_px = int(np.ceil(self.robot_radius / res))
        all_rows, all_cols = [], []

        for r, c in zip(rows, cols):
            # Check if point is inside map bounds
            if 0 <= r < self.map_shape[0] and 0 <= c < self.map_shape[1]:
                # Generate indices of a disk centered at (r, c)
                rr, cc = disk((r, c), radius_px, shape=self.map_shape)
                all_rows.extend(rr)
                all_cols.extend(cc)

        return all_rows, all_cols
```

```
    #RRT* specific functions
    def ball_radius(self):
        #Close neighbor distance
        card_V = len(self.nodes)
        return min(self.gamma_RRT * (np.log(card_V) / card_V ) **
(1.0/2.0), self.epsilon)

    def connect_node_to_point(self, node_i, point_f):
        # #Given two nodes find the non-holonomic path that connects them
        # #Settings
        # #node is a 3 by 1 node
        # #point is a 2 by 1 point
        # print("TO DO: Implement a way to connect two already existing
nodes (for rewiring).")
        return self.simulate_trajectory(node_i, point_f)

    def cost_to_come(self, trajectory_o):
        #The cost to get to a node from lavalle
        p1 = trajectory_o[0:2, 0]
        p2 = trajectory_o[0:2, -1]
        return np.linalg.norm(p1 - p2)

    def update_children(self, node_id):
        # #Given a node_id with a changed cost, update all connected nodes
with the new cost
        # print("TO DO: Update the costs of connected nodes after
rewiring.")

        parent_node = self.nodes[node_id]
        for child_id in parent_node.children_ids:
            child_node = self.nodes[child_id]
            dist = np.linalg.norm(child_node.point[0:2] -
parent_node.point[0:2])
            child_node.cost = parent_node.cost + dist
            self.update_children(child_id)
        return

   ### ADDED HELPER FUNCTION FOR RRT* TO CHECK IF A STRAIGHT LINE PATH IS
COLLISION FREE (USED IN CHOOSE BEST PARENT AND REWIRE STEPS) ###
```

```
    def is_line_collision_free(self, p1, p2):
        # Ensure p1 and p2 are flat 1D arrays for norm and linspace
        p1_flat = p1.flatten()[0:2]
        p2_flat = p2.flatten()[0:2]

        dist = np.linalg.norm(p1_flat - p2_flat)
        res = self.map_settings_dict["resolution"]
        num_samples = int(dist / (res / 2))

        if num_samples < 2:
            return True

        # np.linspace here creates (num_samples, 2).
        # .T turns it into (2, num_samples) which points_to_robot_circle
expects.
        points = np.linspace(p1_flat, p2_flat, num_samples).T

        rr, cc = self.points_to_robot_circle(points)

        # Check if any sampled circle hits an obstacle (0)
        if len(rr) > 0:
            if np.any(self.occupancy_map[rr, cc] == 0):
                return False
        return True
    ###################

    def rrt_planning(self):
        #This function performs RRT on the given map and robot
        #You do not need to demonstrate this function to the TAs, but it
is left in for you to check your work

        for i in range(50000): #Most likely need more iterations than this
to complete the map!
            #Sample map space
            point = self.sample_map_space()
            if self.check_if_duplicate(point):
                continue

            #Get the closest point
            closest_node_id = self.closest_node(point)
```

```
            closest_node = self.nodes[closest_node_id]

            #Simulate driving the robot towards the closest point
            trajectory = self.simulate_trajectory(closest_node.point,
point)

            ########################################
            #Check for collisions
            # print("TO DO: Check for collisions and add safe points to
list of nodes.")
            rr, cc = self.points_to_robot_circle(trajectory[0:2, :])
            if len(rr) == 0 or np.any(self.occupancy_map[rr, cc] == 0):
                continue # Collision detected

            ### ADD NODE
            new_pt = trajectory[:, -1].reshape(3, 1)
            cost = self.nodes[closest_node_id].cost +
self.cost_to_come(trajectory)

            new_node = Node(new_pt, closest_node_id, cost)
            self.nodes.append(new_node)
            closest_node.children_ids.append(len(self.nodes) - 1)

            ### ADD VISUALIZATION
            self.window.add_line(closest_node.point[0:2].flatten(),
new_pt[0:2].flatten(), color=(50, 50, 255))

            if i % 500 == 0:
                dist_to_goal = np.linalg.norm(new_pt[0:2] -
self.goal_point)
                print(f"Iter {i} | Nodes: {len(self.nodes)} | Dist:
{dist_to_goal:.2f}m")

            ##################################################
            # #Check if goal has been reached
            # print("TO DO: Check if at goal point.")
            if np.linalg.norm(new_pt[0:2] - self.goal_point) <
self.stopping_dist:
                print(f"Goal reached at iteration {i}!")
                return self.nodes
```

```
        return self.nodes


    def rrt_star_planning(self):
        #This function performs RRT* for the given map and robot
        for i in range(50000): #Most likely need more iterations than this
to complete the map!
            #Sample
            point = self.sample_map_space()
            if self.check_if_duplicate(point):
                continue

            #Closest Node
            closest_node_id = self.closest_node(point)
            closest_node = self.nodes[closest_node_id]

            #Simulate trajectory
            trajectory = self.simulate_trajectory(closest_node.point,
point)

            ################################
            # #Check for Collision
            # print("TO DO: Check for collision.")
            rr, cc = self.points_to_robot_circle(trajectory[0:2, :])
            if len(rr) == 0 or np.any(self.occupancy_map[rr, cc] == 0):
                continue

            new_pt = trajectory[:, -1].reshape(3, 1)
            dist_moved = self.cost_to_come(trajectory)

            ###FIND NEAR NEIGHBORS
            radius = self.ball_radius()
            near_ids = [idx for idx, node in enumerate(self.nodes)
                        if np.linalg.norm(node.point[0:2] - new_pt[0:2]) <
radius]

            ### CHOOSE BEST PARENT
            best_parent_id = closest_node_id
            min_cost = closest_node.cost + dist_moved
```

```
            for near_id in near_ids:
                near_node = self.nodes[near_id]
                cost_to_near = near_node.cost + np.linalg.norm(new_pt[0:2]
- near_node.point[0:2])

                if cost_to_near < min_cost:
                    if self.is_line_collision_free(near_node.point,
new_pt):
                        best_parent_id = near_id
                        min_cost = cost_to_near

            new_node = Node(new_pt, best_parent_id, min_cost)
            self.nodes.append(new_node)
            new_node_id = len(self.nodes) - 1
            self.nodes[best_parent_id].children_ids.append(new_node_id)

            ####################################
            # #Last node rewire
            # print("TO DO: Last node rewiring")
            # #Close node rewire
            # print("TO DO: Near point rewiring")
            for near_id in near_ids:
                near_node = self.nodes[near_id]
                dist_from_new = np.linalg.norm(near_node.point[0:2] -
new_pt[0:2])

                if new_node.cost + dist_from_new < near_node.cost:
                    if self.is_line_collision_free(new_pt,
near_node.point):
                        if near_node.parent_id != -1:

self.nodes[near_node.parent_id].children_ids.remove(near_id)

                        near_node.parent_id = new_node_id
                        near_node.cost = new_node.cost + dist_from_new
                        new_node.children_ids.append(near_id)
                        self.update_children(near_id)
                        self.window.add_line(new_pt[0:2].flatten(),
near_node.point[0:2].flatten(), color=(0, 255, 255))
```

```
            ### ADD VISUALIZATION

self.window.add_line(self.nodes[best_parent_id].point[0:2].flatten(),
new_pt[0:2].flatten(), color=(50, 50, 255))

            if i % 500 == 0:
                dist_to_goal = np.linalg.norm(new_pt[0:2] -
self.goal_point)
                print(f"Iter {i} | Nodes: {len(self.nodes)} | Goal Dist:
{dist_to_goal:.2f}m")

            ####################################################
            # #Check for early end
            # print("TO DO: Check for early end")
            if np.linalg.norm(new_pt[0:2] - self.goal_point) <
self.stopping_dist:
                print(f"Goal reached at iteration {i}!")
                return self.nodes

    def recover_path(self, node_id = -1):
        path = [self.nodes[node_id].point]
        current_node_id = self.nodes[node_id].parent_id
        while current_node_id > -1:
            path.append(self.nodes[current_node_id].point)
            current_node_id = self.nodes[current_node_id].parent_id
        path.reverse()
        return path

def main():
    start_time = time.time()

    #Set map information
    map_filename = "willowgarageworld_05res.png"
    map_setings_filename = "willowgarageworld_05res.yaml"

    #robot information
    goal_point = np.array([[42.05], [-44]]) #m
    stopping_dist = 0.5 #m
```

```
    path_planner = PathPlanner(map_filename, map_setings_filename,
goal_point, stopping_dist)
    nodes = path_planner.rrt_star_planning()
    node_path_metric = np.hstack(path_planner.recover_path())

    path_list = path_planner.recover_path()
    if path_list:
        total_dist = 0
        for i in range(len(path_list) - 1):
            p1 = path_list[i][0:2]
            p2 = path_list[i+1][0:2]
            total_dist += np.linalg.norm(p1 - p2)
        print(f"Total Path Distance: {total_dist:.2f} meters")

        node_path_metric = np.hstack(path_list)
        np.save("shortest_path_rrt_star_CZ1.npy", node_path_metric)
        print(f"Path saved. Duration: {time.time() - start_time:.2f}s")

    # Keep visualizer open
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT: running = False
        time.sleep(0.05)
    pygame.quit()


if __name__ == '__main__':
    main()
```

l2_follow_path.py

```
#!/usr/bin/env python3
from __future__ import division, print_function
import os

import numpy as np
from scipy.linalg import block_diag
from scipy.spatial.distance import cityblock
import rospy
import tf2_ros

# msgs
from geometry_msgs.msg import TransformStamped, Twist, PoseStamped
from nav_msgs.msg import Path, Odometry, OccupancyGrid
from visualization_msgs.msg import Marker

# ros and se2 conversion utils
import utils


TRANS_GOAL_TOL = .3  # m, tolerance to consider a goal complete
ROT_GOAL_TOL = .5 # rad, tolerance to consider a goal complete
TRANS_VEL_OPTS = [0, 0.05, 0.15, 0.26] # m/s, max of real robot is .26
ROT_VEL_OPTS = np.linspace(-1.82, 1.82, 11) # rad/s, max of real robot is
1.82
CONTROL_RATE = 5 # Hz, how frequently control signals are sent
CONTROL_HORIZON = 5  # seconds. if this is set too high and INTEGRATION_DT
is too low, code will take a long time to run!
INTEGRATION_DT = .05  # s, delta t to propagate trajectories forward by #
Increased step size slightly to speed up rollout math
COLLISION_RADIUS = 0.225 # m, radius from base_link to use for collisions,
min of 0.2077 based on dimensions of .281 x .306
ROT_DIST_MULT = 0.05 # multiplier to change effect of rotational distance
in choosing correct control # Lowered to prioritize forward progress
OBS_DIST_MULT = .1  # multiplier to change the effect of low distance to
obstacles on a path
MIN_TRANS_DIST_TO_USE_ROT = 0.4  # m, robot has to be within this distance
to use rot distance in cost
PATH_NAME = 'MYHAL_shortest_path_rrt_CZ.npy'
```

```
LOOK_AHEAD_DIST = 0.8 # m, how far ahead to look for the next node

# here are some hardcoded paths to use if you want to develop l2_planning
and this file in parallel
# TEMP_HARDCODE_PATH = [[2, 0, 0], [2.75, -1, -np.pi/2], [2.75, -4,
-np.pi/2], [2, -4.4, np.pi]]  # almost collision-free
TEMP_HARDCODE_PATH = [[2, -.5, 0], [2.4, -1, -np.pi/2], [2.45, -3.5,
-np.pi/2], [1.5, -4.4, np.pi]]  # some possible collisions


class PathFollower():
    def __init__(self):
        # time full path
        self.path_follow_start_time = rospy.Time.now()

        # use tf2 buffer to access transforms between existing frames in
tf tree
        self.tf_buffer = tf2_ros.Buffer()
        self.listener = tf2_ros.TransformListener(self.tf_buffer)
        rospy.sleep(1.0)  # time to get buffer running

        # constant transforms
        self.map_odom_tf = self.tf_buffer.lookup_transform('map', 'odom',
rospy.Time(0), rospy.Duration(2.0)).transform
        print(self.map_odom_tf)

        # subscribers and publishers
        self.cmd_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=1)
        self.global_path_pub = rospy.Publisher('~global_path', Path,
queue_size=1, latch=True)
        self.local_path_pub = rospy.Publisher('~local_path', Path,
queue_size=1)
        self.collision_marker_pub = rospy.Publisher('~collision_marker',
Marker, queue_size=1)

        # map
        map = rospy.wait_for_message('/map', OccupancyGrid)
        self.map_np = np.array(map.data).reshape(map.info.height,
map.info.width)
        self.map_resolution = round(map.info.resolution, 5)
```

```
        self.map_origin = -utils.se2_pose_from_pose(map.info.origin)  #
negative because of weird way origin is stored
        print(self.map_origin)
        self.map_nonzero_idxes = np.argwhere(self.map_np)
        print(map)


        # collisions
        self.collision_radius_pix = COLLISION_RADIUS / self.map_resolution
        self.collision_marker = Marker()
        self.collision_marker.header.frame_id = '/map'
        self.collision_marker.ns = '/collision_radius'
        self.collision_marker.id = 0
        self.collision_marker.type = Marker.CYLINDER
        self.collision_marker.action = Marker.ADD
        self.collision_marker.scale.x = COLLISION_RADIUS * 2
        self.collision_marker.scale.y = COLLISION_RADIUS * 2
        self.collision_marker.scale.z = 1.0
        self.collision_marker.color.g = 1.0
        self.collision_marker.color.a = 0.5

        # transforms
        self.map_baselink_tf = self.tf_buffer.lookup_transform('map',
'base_link', rospy.Time(0), rospy.Duration(2.0))
        self.pose_in_map_np = np.zeros(3)
        self.pos_in_map_pix = np.zeros(2)
        self.update_pose()

        # path variables
        cur_dir = os.path.dirname(os.path.realpath(__file__))

        # to use the temp hardcoded paths above, switch the comment on the
following two lines
        self.path_tuples = np.load(os.path.join(cur_dir, 'path.npy')).T
        # self.path_tuples = np.array(TEMP_HARDCODE_PATH)

        self.path = utils.se2_pose_list_to_path(self.path_tuples, 'map')
        self.global_path_pub.publish(self.path)

        # goal
```

```
        self.cur_goal = np.array(self.path_tuples[0])
        self.cur_path_index = 0

        # trajectory rollout tools
        # self.all_opts is a Nx2 array with all N possible combinations of
the t and v vels, scaled by integration dt
        self.all_opts = np.array(np.meshgrid(TRANS_VEL_OPTS,
ROT_VEL_OPTS)).T.reshape(-1, 2)

        # if there is a [0, 0] option, remove it
        all_zeros_index = (np.abs(self.all_opts) < [0.001,
0.001]).all(axis=1).nonzero()[0]
        if all_zeros_index.size > 0:
            self.all_opts = np.delete(self.all_opts, all_zeros_index,
axis=0)
        self.all_opts_scaled = self.all_opts * INTEGRATION_DT

        self.num_opts = self.all_opts_scaled.shape[0]
        self.horizon_timesteps = int(np.ceil(CONTROL_HORIZON /
INTEGRATION_DT))

        self.rate = rospy.Rate(CONTROL_RATE)

        rospy.on_shutdown(self.stop_robot_on_shutdown)
        self.follow_path()

    def follow_path(self):
        while not rospy.is_shutdown():
            # timing for debugging...loop time should be less than
1/CONTROL_RATE
            tic = rospy.Time.now()

            self.update_pose()
            self.check_and_update_goal()

            # start trajectory rollout algorithm
            local_paths = np.zeros([self.horizon_timesteps + 1,
self.num_opts, 3])
            local_paths[0] =
np.atleast_2d(self.pose_in_map_np).repeat(self.num_opts, axis=0)
```

```
            # print("TO DO: Propogate the trajectory forward, storing the
resulting points in local_paths!")
            # Grab linear (v) and angular (w) velocities from our
predefined control options
            v = self.all_opts[:, 0]
            w = self.all_opts[:, 1]

            for t in range(1, self.horizon_timesteps + 1):
                # propogate trajectory forward, assuming perfect control
of velocity and no dynamic effects
                # Step through time to predict where the robot goes for
each v/w combination
                prev_pose = local_paths[t-1]
                theta = prev_pose[:, 2]
                # Update (x, y, theta) using the Unicycle Kinematics model
                local_paths[t, :, 0] = prev_pose[:, 0] + v * np.cos(theta)
* INTEGRATION_DT
                local_paths[t, :, 1] = prev_pose[:, 1] + v * np.sin(theta)
* INTEGRATION_DT
                local_paths[t, :, 2] = theta + w * INTEGRATION_DT


            # check all trajectory points for collisions
            # first find the closest collision point in the map to each
local path point
            local_paths_pixels = (self.map_origin[:2] + local_paths[:, :,
:2]) / self.map_resolution
            map_h, map_w = self.map_np.shape
            valid_opts = []

            #print("TO DO: Check the points in local_path_pixels for
collisions")
            # remove trajectories that were deemed to have collisions
            #print("TO DO: Remove trajectories with collisions!")
            for opt in range(self.num_opts):
                # Check the trajectory doesn't go outside the bounds of
the map image
                traj_pix = local_paths_pixels[:, opt, :].astype(int)
```

```
                if np.any(traj_pix[:, 0] < 0) or np.any(traj_pix[:, 0] >=
map_w) or \
                   np.any(traj_pix[:, 1] < 0) or np.any(traj_pix[:, 1] >=
map_h):
                    continue
                # Check for obstacles -  values > 50 usually indicate a
wall or obstacle
                if np.any(self.map_np[traj_pix[:, 1], traj_pix[:, 0]] >
50):
                    continue
                valid_opts.append(opt)


            # calculate final cost and choose best option
            # print("TO DO: Calculate the final cost and choose the best
control option!")
            if not valid_opts:
                rospy.logwarn_throttle(1, "[WARN] All paths colliding!
Safety Stop.")
                control = [0, 0]
            else:
                costs = []
                for opt in valid_opts:
                    # Look at the final predicted position of the
trajectory
                    end_pose = local_paths[-1, opt]

                    # Look at regular and angular distance
                    dist = np.linalg.norm(end_pose[:2] -
self.cur_goal[:2])
                    abs_diff = np.abs(end_pose[2] - self.cur_goal[2])
                    rot_dist = min(np.pi * 2 - abs_diff, abs_diff)

                   # Care more about linear distance, only care about
angular distance when we are close to the node
                    cost = dist
                    if dist < MIN_TRANS_DIST_TO_USE_ROT:
                        cost += rot_dist * ROT_DIST_MULT
                    costs.append(cost)
```

```
                best_opt = valid_opts[np.argmin(costs)]
                control = self.all_opts[best_opt]

self.local_path_pub.publish(utils.se2_pose_list_to_path(local_paths[:,
best_opt], 'map'))

            dist_to_goal = np.linalg.norm(self.pose_in_map_np[:2] -
self.cur_goal[:2])
            print(f"Targeting Node {self.cur_path_index} | Dist:
{dist_to_goal:.2f}m | Safe Paths: {len(valid_opts)}/{self.num_opts} | Vel:
{control[0]} m/s")

            self.cmd_pub.publish(utils.unicyle_vel_to_twist(control))
            self.rate.sleep()

    def update_pose(self):
        try:
            self.map_baselink_tf = self.tf_buffer.lookup_transform('map',
'base_link', rospy.Time(0)).transform
            self.pose_in_map_np[:] = [self.map_baselink_tf.translation.x,
self.map_baselink_tf.translation.y,

utils.euler_from_ros_quat(self.map_baselink_tf.rotation)[2]]
        except: pass


    def check_and_update_goal(self):
        # LOOK-AHEAD LOGIC: Scan path for furthest node within radius
        found_further = False
        for i in range(self.cur_path_index, len(self.path_tuples)):
            dist_to_node = np.linalg.norm(self.pose_in_map_np[:2] -
self.path_tuples[i][:2])
            if dist_to_node < LOOK_AHEAD_DIST:
                if i > self.cur_path_index:
                    self.cur_path_index = i
                    self.cur_goal = np.array(self.path_tuples[i])
                    found_further = True

        if found_further:
            print(f">>> SKIPPED AHEAD to Node {self.cur_path_index}")
```

```
        # Completion check
        dist_from_final = np.linalg.norm(self.pose_in_map_np[:2] -
self.path_tuples[-1][:2])
        if dist_from_final < TRANS_GOAL_TOL:
            print(f"\n[SUCCESS] Final waypoint reached within
{TRANS_GOAL_TOL}m threshold.")
            rospy.loginfo("FINAL GOAL REACHED!")
            rospy.signal_shutdown("Path Complete")


    def stop_robot_on_shutdown(self):
        print("[SHUTDOWN] Stopping robot motors.")
        self.cmd_pub.publish(Twist())


if __name__ == '__main__':
    try:
        rospy.init_node('path_follower', log_level=rospy.INFO)
        pf = PathFollower()
    except rospy.ROSInterruptException: pass
```

l2_follow_path_myhal l2_follow_path_myhal.py

```
#!/usr/bin/env python3
from __future__ import division, print_function
import os


import numpy as np
from scipy.linalg import block_diag
from scipy.spatial.distance import cityblock
import rospy
import tf2_ros


# msgs
from geometry_msgs.msg import TransformStamped, Twist, PoseStamped
from nav_msgs.msg import Path, Odometry, OccupancyGrid
```

```
from visualization_msgs.msg import Marker


# ros and se2 conversion utils
import utils




TRANS_GOAL_TOL = .1  # m, tolerance to consider a goal complete
ROT_GOAL_TOL = .3  # rad, tolerance to consider a goal complete
TRANS_VEL_OPTS = [0, 0.025, 0.13, 0.26]  # m/s, max of real robot is .26
ROT_VEL_OPTS = np.linspace(-1.82, 1.82, 11)  # rad/s, max of real robot is
1.82
CONTROL_RATE = 5  # Hz, how frequently control signals are sent
CONTROL_HORIZON = 5  # seconds. if this is set too high and INTEGRATION_DT
is too low, code will take a long time to run!
INTEGRATION_DT = .025  # s, delta t to propagate trajectories forward by
COLLISION_RADIUS = 0.225  # m, radius from base_link to use for
collisions, min of 0.2077 based on dimensions of .281 x .306
ROT_DIST_MULT = .1  # multiplier to change effect of rotational distance
in choosing correct control
OBS_DIST_MULT = .1  # multiplier to change the effect of low distance to
obstacles on a path
MIN_TRANS_DIST_TO_USE_ROT = TRANS_GOAL_TOL  # m, robot has to be within
this distance to use rot distance in cost
PATH_NAME = 'MYHAL_shortest_path_rrt_CZ.npy'  # saved path from
l2_planning.py, should be in the same directory as this file


# here are some hardcoded paths to use if you want to develop l2_planning
and this file in parallel
# TEMP_HARDCODE_PATH = [[2, 0, 0], [2.75, -1, -np.pi/2], [2.75, -4,
-np.pi/2], [2, -4.4, np.pi]]  # almost collision-free
TEMP_HARDCODE_PATH = [[2, -.5, 0], [2.4, -1, -np.pi/2], [2.45, -3.5,
-np.pi/2], [1.5, -4.4, np.pi]]  # some possible collisions
```

```
#Map Handling Functions
def load_map(filename):
    import matplotlib.image as mpimg
    import cv2
    im = cv2.imread("../maps/" + filename)
    im = cv2.flip(im, 0)
    # im = mpimg.imread("../maps/" + filename)
    if len(im.shape) > 2:
        im = im[:,:,0]
    im_np = np.array(im)  #Whitespace is true, black is false
    im_np = np.logical_not(im_np)     #for ros
    return im_np


class PathFollower():
    def __init__(self):
        # time full path
        self.path_follow_start_time = rospy.Time.now()


        # use tf2 buffer to access transforms between existing frames in
tf tree
        self.tf_buffer = tf2_ros.Buffer()
        self.listener = tf2_ros.TransformListener(self.tf_buffer)
        rospy.sleep(1.0)  # time to get buffer running


        # constant transforms
        self.map_odom_tf = self.tf_buffer.lookup_transform('map', 'odom',
rospy.Time(0), rospy.Duration(2.0)).transform
        print(self.map_odom_tf)


        # subscribers and publishers
        self.cmd_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=1)
        self.global_path_pub = rospy.Publisher('~global_path', Path,
queue_size=1, latch=True)
        self.local_path_pub = rospy.Publisher('~local_path', Path,
queue_size=1)
```

```
        self.collision_marker_pub = rospy.Publisher('~collision_marker',
Marker, queue_size=1)


        # map
        # map = rospy.wait_for_message('/map', OccupancyGrid)
        # self.map_np = np.array(map.data).reshape(map.info.height,
map.info.width)
        # self.map_resolution = round(map.info.resolution, 5)
        # self.map_origin = -utils.se2_pose_from_pose(map.info.origin)  #
negative because of weird way origin is stored
        # self.map_nonzero_idxes = np.argwhere(self.map_np)
        map_filename = "myhal.png"
        occupancy_map = load_map(map_filename)
        self.map_np = occupancy_map
        self.map_resolution = 0.05
        self.map_origin = np.array([ 0.2 , 0.2 ,-0. ])
        self.map_nonzero_idxes = np.argwhere(self.map_np)




        # collisions
        self.collision_radius_pix = COLLISION_RADIUS / self.map_resolution
        self.collision_marker = Marker()
        self.collision_marker.header.frame_id = '/map'
        self.collision_marker.ns = '/collision_radius'
        self.collision_marker.id = 0
        self.collision_marker.type = Marker.CYLINDER
        self.collision_marker.action = Marker.ADD
        self.collision_marker.scale.x = COLLISION_RADIUS * 2
        self.collision_marker.scale.y = COLLISION_RADIUS * 2
        self.collision_marker.scale.z = 1.0
        self.collision_marker.color.g = 1.0
        self.collision_marker.color.a = 0.5


        # transforms
        self.map_baselink_tf = self.tf_buffer.lookup_transform('map',
'base_footprint', rospy.Time(0), rospy.Duration(2.0))
```

```
        self.pose_in_map_np = np.zeros(3)
        self.pos_in_map_pix = np.zeros(2)
        self.update_pose()


        # path variables
        cur_dir = os.path.dirname(os.path.realpath(__file__))


        # to use the temp hardcoded paths above, switch the comment on the
following two lines
        self.path_tuples = np.load(os.path.join(cur_dir,
'shortest_path_rrt_CZ.npy')).T
        # self.path_tuples = np.array(TEMP_HARDCODE_PATH)


        self.path = utils.se2_pose_list_to_path(self.path_tuples, 'map')
        self.global_path_pub.publish(self.path)


        # goal
        self.cur_goal = np.array(self.path_tuples[0])
        self.cur_path_index = 0


        # trajectory rollout tools
        # self.all_opts is a Nx2 array with all N possible combinations of
the t and v vels, scaled by integration dt
        self.all_opts = np.array(np.meshgrid(TRANS_VEL_OPTS,
ROT_VEL_OPTS)).T.reshape(-1, 2)


        # if there is a [0, 0] option, remove it
        all_zeros_index = (np.abs(self.all_opts) < [0.001,
0.001]).all(axis=1).nonzero()[0]
        if all_zeros_index.size > 0:
            self.all_opts = np.delete(self.all_opts, all_zeros_index,
axis=0)
        self.all_opts_scaled = self.all_opts * INTEGRATION_DT
```

```
        self.num_opts = self.all_opts_scaled.shape[0]
        self.horizon_timesteps = int(np.ceil(CONTROL_HORIZON /
INTEGRATION_DT))


        self.rate = rospy.Rate(CONTROL_RATE)


        rospy.on_shutdown(self.stop_robot_on_shutdown)
        self.follow_path()


    def follow_path(self):
        while not rospy.is_shutdown():
            tic = rospy.Time.now()
            self.update_pose()
            self.check_and_update_goal()

            local_paths = np.zeros([self.horizon_timesteps + 1,
self.num_opts, 3])
            local_paths[0] =
np.atleast_2d(self.pose_in_map_np).repeat(self.num_opts, axis=0)


            v = self.all_opts[:, 0]
            w = self.all_opts[:, 1]


            for t in range(1, self.horizon_timesteps + 1):
                prev_pose = local_paths[t-1]
                theta = prev_pose[:, 2]
                local_paths[t, :, 0] = prev_pose[:, 0] + v * np.cos(theta)
* INTEGRATION_DT
                local_paths[t, :, 1] = prev_pose[:, 1] + v * np.sin(theta)
* INTEGRATION_DT
                local_paths[t, :, 2] = theta + w * INTEGRATION_DT

            local_paths_pixels = (self.map_origin[:2] + local_paths[:, :,
:2]) / self.map_resolution
```

```
            map_h, map_w = self.map_np.shape
            valid_opts = []


            for opt in range(self.num_opts):
                traj_pix = local_paths_pixels[:, opt, :].astype(int)
                if np.any(traj_pix[:, 0] < 0) or np.any(traj_pix[:, 0] >=
map_w) or \
                   np.any(traj_pix[:, 1] < 0) or np.any(traj_pix[:, 1] >=
map_h):
                    continue
                if np.any(self.map_np[traj_pix[:, 1], traj_pix[:, 0]] >
50):
                    continue
                valid_opts.append(opt)

            if not valid_opts:
                rospy.logwarn_throttle(1, "[WARN] All paths colliding!
Safety Stop.")
                control = [0, 0]
            else:
                costs = []
                for opt in valid_opts:
                    end_pose = local_paths[-1, opt]
                    dist = np.linalg.norm(end_pose[:2] -
self.cur_goal[:2])

                    abs_diff = np.abs(end_pose[2] - self.cur_goal[2])
                    rot_dist = min(np.pi * 2 - abs_diff, abs_diff)

                    cost = dist
                    if dist < MIN_TRANS_DIST_TO_USE_ROT:
                        cost += rot_dist * ROT_DIST_MULT
                    costs.append(cost)

                best_opt = valid_opts[np.argmin(costs)]
                control = self.all_opts[best_opt]

self.local_path_pub.publish(utils.se2_pose_list_to_path(local_paths[:,
best_opt], 'map'))
```

```
            dist_to_goal = np.linalg.norm(self.pose_in_map_np[:2] -
self.cur_goal[:2])
            print(f"Targeting Node {self.cur_path_index} | Dist:
{dist_to_goal:.2f}m | Safe Paths: {len(valid_opts)}/{self.num_opts} | Vel:
{control[0]} m/s")


            self.cmd_pub.publish(utils.unicyle_vel_to_twist(control))
            self.rate.sleep()


    def update_pose(self):
        # Update numpy poses with current pose using the tf_buffer
        self.map_baselink_tf = self.tf_buffer.lookup_transform('map',
'base_footprint', rospy.Time(0)).transform
        self.pose_in_map_np[:] = [self.map_baselink_tf.translation.x,
self.map_baselink_tf.translation.y,

utils.euler_from_ros_quat(self.map_baselink_tf.rotation)[2]]
        self.pos_in_map_pix = (self.map_origin[:2] +
self.pose_in_map_np[:2]) / self.map_resolution
        self.collision_marker.header.stamp = rospy.Time.now()
        self.collision_marker.pose =
utils.pose_from_se2_pose(self.pose_in_map_np)
        self.collision_marker_pub.publish(self.collision_marker)


    def check_and_update_goal(self):
        # iterate the goal if necessary
        dist_from_goal = np.linalg.norm(self.pose_in_map_np[:2] -
self.cur_goal[:2])
        abs_angle_diff = np.abs(self.pose_in_map_np[2] - self.cur_goal[2])
        rot_dist_from_goal = min(np.pi * 2 - abs_angle_diff,
abs_angle_diff)
        if dist_from_goal < TRANS_GOAL_TOL and rot_dist_from_goal <
ROT_GOAL_TOL:
            rospy.loginfo("Goal {goal} at {pose} complete.".format(
                    goal=self.cur_path_index, pose=self.cur_goal))
            if self.cur_path_index == len(self.path_tuples) - 1:
```

```
                rospy.loginfo("Full path complete in {time}s! Path
Follower node shutting down.".format(
                    time=(rospy.Time.now() -
self.path_follow_start_time).to_sec()))
                rospy.signal_shutdown("Full path complete! Path Follower
node shutting down.")
            else:
                self.cur_path_index += 1
                self.cur_goal =
np.array(self.path_tuples[self.cur_path_index])
        else:
            rospy.logdebug("Goal {goal} at {pose}, trans error: {t_err},
rot error: {r_err}.".format(
                goal=self.cur_path_index, pose=self.cur_goal,
t_err=dist_from_goal, r_err=rot_dist_from_goal
            ))


    def stop_robot_on_shutdown(self):
        self.cmd_pub.publish(Twist())
        rospy.loginfo("Published zero vel on shutdown.")




if __name__ == '__main__':
    try:
        rospy.init_node('path_follower', log_level=rospy.DEBUG)
        pf = PathFollower()
    except rospy.ROSInterruptException:
        pass
```